



UNIVERSITY OF PETROLEUM AND ENERGY
STUDIES
DEHRADUN

Applied DevOps

Experiment 4

MTECH - School of Computer Science
CYBER SECURITY AND FORENSICS

Submitted To: Dr. Krishnaraj

Name: Jigesh Sheoran

SAP ID: 590025428

Objective — To practically study the history, architecture, basic operations, and feature differences of three types of Version Control Systems using RCS (Local VCS), Subversion (Centralized VCS), and Mercurial (Distributed VCS).

Task to be Performed

PART A – Understanding Historical Evolution

1. The evolution of Version Control Systems: Local, Centralized, and Distributed.

Version control tracks changes to code or files over time. Three models have emerged — each fixing the problems of the last one.

1.1 Local VCS (1970s – early 1980s)

Developers kept copies of old files in separate folders. Tools like RCS and SCCS automated this by saving file differences (deltas) in a local database on the same machine.

- One developer, one machine — no sharing at all.
- Accidental delete of the local DB = all history lost.

1.2 Centralized VCS (1980s – 2000s)

Teams needed collaboration. CVCS moved the repository to a single shared server. Developers checked out files, made changes, then committed back. CVS (1986) and SVN (2000) are classic examples.

- Everyone could see and share work — a big step forward.
- But the server became a Single Point of Failure (SPOF).

1.3 Distributed VCS (2000s – present)

DVCS (Git 2005, Mercurial 2005) gave every developer a full copy of the repository — history and all. No central server is strictly required, so the system is resilient and fast.

- Works offline. No server, no problem.
- Cheap branching and merging. Peer-to-peer sharing.
-

2. Architectural differences between the three models.

Model 1 - Local VCS (e.g., RCS, SCCS)

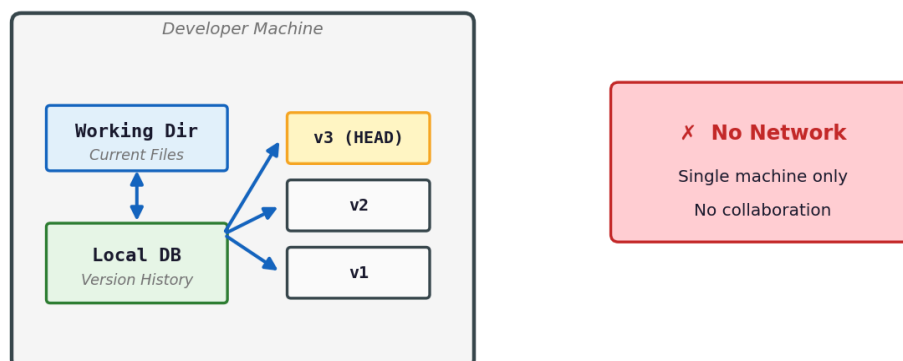


Figure 1 – Local VCS: all data on one machine

Model 2 - Centralized VCS (e.g., SVN, CVS)

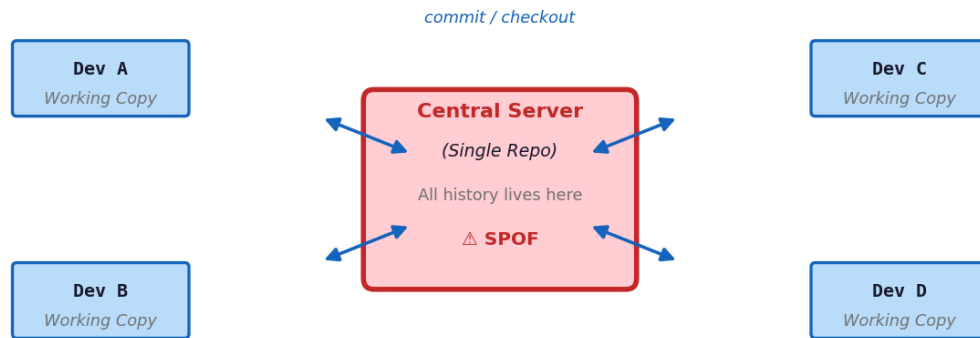


Figure 2 – Centralized VCS: single authoritative server

Model 3 - Distributed VCS (e.g., Git, Mercurial)

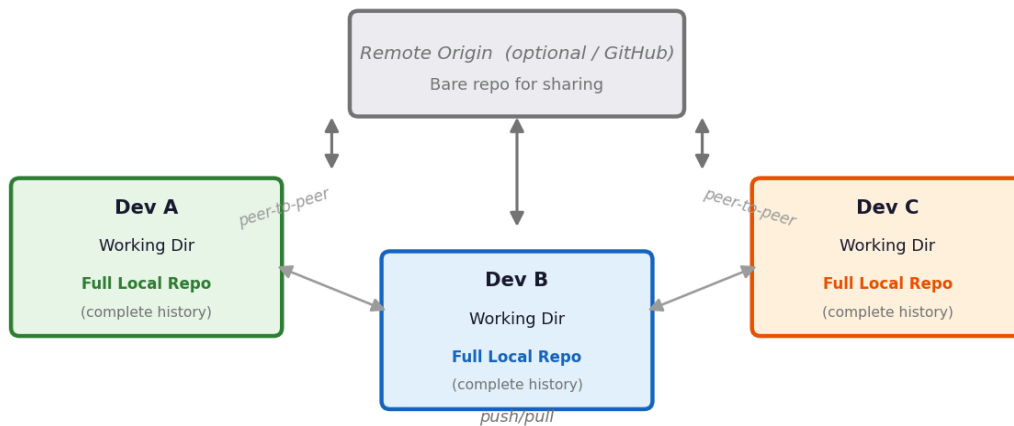


Figure 3 – Distributed VCS: every developer has full repo

Comparison Summary

Aspect	Local	Centralized	Distributed
Repo Location	One machine	Central server	Every developer
Network needed?	No	Yes (always)	No (mostly)
Collaboration	None	Via server	Peer-to-peer
SPOF risk	Machine crash	Server down = halt	No single SPOF
Example tools	RCS, SCCS	SVN, CVS	Git, Mercurial

3. Limitations of each model that led to the development of the next.

3.1 Why Local VCS was not enough

- No sharing — two developers could not work on same project at same time.
- All data on one machine — hardware failure wiped all history.
- No concept of a "latest version" across a team.

→ *These problems forced the move to a central server.*

3.2 Why Centralized VCS was not enough

- Server down = entire team blocked. Nobody can commit or get history.
- All commits go through the network — slow over poor connections.
- Branching was expensive and painful; teams avoided it.
- Single server = single point of data loss. Backup discipline required.

→ *These problems drove development of distributed systems.*

3.3 Current limits of Distributed VCS

- More complex mental model — cloning, remotes, rebasing confuse beginners.
- Binary files (images, videos) inflate repo size — Git LFS is a workaround.
- Access control per-folder is harder compared to SVN (which controls by path).

→ *DVCS is the modern standard but continues to evolve (e.g., Git bundles, partial clones).*

PART B – Local Version Control Using RCS

Task 1: Create a directory named 'VCS_Experiment'. Create a Python file 'app.py' with: `print('Version 1')`

Step 1: Install RCS

```
[sheoraninfosec@Jigeshs-MacBook-Air ~ % brew install rcs  
==> Auto-updating Homebrew...
```

```
[sheoraninfosec@Jigeshs-MacBook-Air ~ % mkdir VCS-exp  
[sheoraninfosec@Jigeshs-MacBook-Air ~ % cd VCS-exp  
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % nano app.py  
sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % █
```

```
UW PICO 5.09 File: app.py  
print("Version 1")█
```

Task 2: Using RCS:

Step 1: Put under version control

RCS workflow is:

1. **rscs -i** → initializes (no versions yet)
2. **ci** → **creates first revision (1.1)**
3. **co** → used AFTER first commit

```
VCS-exp --zsh -- 113x43
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % ls
app.py RCS
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % rcs -i app.py
RCS file: RCS/app.py,v
enter description, terminated with single '.' or end of file:
NOTE: This is NOT the log message!
>> Experiment 4
>> .
done
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % co -l app.py
RCS/app.py,v --> app.py
no revisions present; generating empty revision 0.0
co: warning: no revisions, so nothing can be locked
writable app.py exists; remove it? [nyl(n): n
co: checkout aborted
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % ci -u app.py
RCS/app.py,v <-- app.py
initial revision: 1.1
done
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % co -l app.py
RCS/app.py,v --> app.py
revision 1.1 (locked)
done
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % rlog app.py

RCS file: RCS/app.py,v
Working file: app.py
head: 1.1
branch:
locks: strict
    sheoraninfosec: 1.1
access list:
symbolic names:
keyword substitution: kv
total revisions: 1;    selected revisions: 1
description:
Experiment 4
=====
revision 1.1    locked by: sheoraninfosec;
date: 2026/04/03 18:52:00;  author: sheoraninfosec;  state: Exp;
Initial revision
=====
sheoraninfosec@Jigeshs-MacBook-Air VCS-exp %
```

2. Modify to Version 2 and commit.

```
UW PICO 5.09                                     File: app.py

print("Version 2")

[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % nano app.py
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % ci -u app.py
RCS/app.py,v <-- app.py
new revision: 1.2; previous revision: 1.1
enter log message, terminated with single '.' or end of file:
>> Updated to Version 2
>> .
done
sheoraninfosec@Jigeshs-MacBook-Air VCS-exp %
```

3. Modify to Version 3 and commit.

```
UW PICO 5.09 File: app.py

print("Version 3")

[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % nano app.py
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % ci -u app.py
RCS/app.py,v <-- app.py
new revision: 1.3; previous revision: 1.2
enter log message, terminated with single '.' or end of file:
>> Updated to Version 3
>> .
done
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % co -l app.py
RCS/app.py,v --> app.py
revision 1.3 (locked)
done
sheoraninfosec@Jigeshs-MacBook-Air VCS-exp %
```

4. Display revision history.

```
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % rlog app.py

RCS file: RCS/app.py,v
Working file: app.py
head: 1.3
branch:
locks: strict
    sheoraninfosec: 1.3
access list:
symbolic names:
keyword substitution: kv
total revisions: 3;    selected revisions: 3
description:
Experiment 4
-----
revision 1.3    locked by: sheoraninfosec;
date: 2026/04/03 19:03:36;  author: sheoraninfosec;  state: Exp;  lines: +2 -1
Updated to Version 3
-----
revision 1.2
date: 2026/04/03 18:56:24;  author: sheoraninfosec;  state: Exp;  lines: +1 -1
Updated to Version 2
-----
revision 1.1
date: 2026/04/03 18:52:00;  author: sheoraninfosec;  state: Exp;
Initial revision
=====
sheoraninfosec@Jigeshs-MacBook-Air VCS-exp %
```

5. Retrieve Version 1.

```
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % co -r1.1 app.py
RCS/app.py,v --> app.py
revision 1.1
writable app.py exists; remove it? [ny](n): y
done
sheoraninfosec@Jigeshs-MacBook-Air VCS-exp %
```

This will:

- Delete current file (Version 3)
- Replace it with Version 1

Verification:

```
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % cat app.py
print("Version 1")
sheoraninfosec@Jigeshs-MacBook-Air VCS-exp %
```

Task 3: Analyze storage mechanism, collaboration capability, repository structure, and limitations.

1. Storage Mechanism

- RCS stores all revisions in a single file inside the **RCS** / directory.
- The file is stored with a **,v** extension (e.g., **app.py,v**).
- It uses a **reverse delta storage mechanism**, where:
 - Latest version is stored fully
 - Previous versions are stored as differences (deltas)
- This approach reduces storage space but makes reconstruction of older versions slower.

2. Collaboration Capability

- RCS is designed for **single-user environments**.
- It does not support:
 - Multi-user access
 - Remote repositories
 - Concurrent editing
- Uses a **file locking mechanism (co -l)** to prevent conflicts.
- Collaboration is practically **non-existent**, making it unsuitable for team projects.

3. Repository Structure

- Repository is not centralized or distributed.
- It is **local and file-based**, stored within:

RCS/app.py,v

- Each file has its own independent version history.
- There is **no concept of project-level repository**, unlike modern systems.

4. Limitations

- **✗** No support for team collaboration
- **✗** File-level tracking only (not project-level)
- **✗** Manual locking required before editing
- **✗** Poor usability and complex workflow
- **✗** No branching and merging capabilities
- **✗** No remote access or networking support

PART C – Centralized Version Control Using Subversion (SVN)

Task 4: Create SVN repository with trunk, branches, and tags. Checkout trunk.

1. Create SVN repository

```
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % svnadmin create myrepo  
sheoraninfosec@Jigeshs-MacBook-Air VCS-exp %
```

2. Create project structure

```
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % svnadmin create myrepo  
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % mkdir svn  
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % cd svn  
[sheoraninfosec@Jigeshs-MacBook-Air svn % svn checkout file://$PWD/../myrepo repo_wc  
svn: E125002: URL 'file:///Users/sheoraninfosec/VCS-exp/svn/../myrepo' contains a '..' element  
[sheoraninfosec@Jigeshs-MacBook-Air svn % cd repo_wc  
cd: no such file or directory: repo_wc  
[sheoraninfosec@Jigeshs-MacBook-Air svn % pwd  
/Users/sheoraninfosec/VCS-exp/svn  
[sheoraninfosec@Jigeshs-MacBook-Air svn % svn checkout file:///Users/sheoraninfosec/VCS-exp/myrepo repo_wc  
Checked out revision 0.  
[sheoraninfosec@Jigeshs-MacBook-Air svn % cd repo_wc  
sheoraninfosec@Jigeshs-MacBook-Air repo_wc %
```

3. Create trunk, branches, tags

```
[sheoraninfosec@Jigeshs-MacBook-Air svn % cd repo_wc  
[sheoraninfosec@Jigeshs-MacBook-Air repo_wc % mkdir trunk branches tags  
[sheoraninfosec@Jigeshs-MacBook-Air repo_wc % svn add trunk branches tags  
A      trunk  
A      branches  
A      tags  
[sheoraninfosec@Jigeshs-MacBook-Air repo_wc % svn commit -m "Initial repository structure"  
Adding      branches  
Adding      tags  
Adding      trunk  
Committing transaction...  
Committed revision 1.  
sheoraninfosec@Jigeshs-MacBook-Air repo_wc %
```

4. Checkout trunk

```
[sheoraninfosec@Jigeshs-MacBook-Air repo_wc % cd trunk  
sheoraninfosec@Jigeshs-MacBook-Air trunk %
```

TASK 5

1. Add & commit Version 1

```
UW PICO 5.09                                     File: app.py  
  
print("Version 1")
```

```
[sheoraninfosec@Jigeshs-MacBook-Air trunk % nano app.py  
[sheoraninfosec@Jigeshs-MacBook-Air trunk % svn add app.py  
A      app.py  
[sheoraninfosec@Jigeshs-MacBook-Air trunk % svn commit -m "Version 1"  
zsh: command not found: svn  
[sheoraninfosec@Jigeshs-MacBook-Air trunk % svn commit -m "Version 1"  
Adding      app.py  
Transmitting file data .done  
Committing transaction...  
Committed revision 2.  
sheoraninfosec@Jigeshs-MacBook-Air trunk %
```

2. Modify & Commit Version 2

```
UW PICO 5.09 File: app.py
print("Version 2")
```

```
[sheoraninfosec@Jigeshs-MacBook-Air trunk % nano app.py
[sheoraninfosec@Jigeshs-MacBook-Air trunk % svn commit -m "version 2"
Sending      app.py
Transmitting file data .done
Committing transaction...
Committed revision 3.
sheoraninfosec@Jigeshs-MacBook-Air trunk %
```

```
sheoraninfosec@Jigeshs-MacBook-Air trunk % nano app.py
```

```
UW PICO 5.09 File: app.py
print("Version 3")
```

```
[sheoraninfosec@Jigeshs-MacBook-Air trunk % svn commit -m "Version 3"
Sending      app.py
Transmitting file data .done
Committing transaction...
Committed revision 4.
sheoraninfosec@Jigeshs-MacBook-Air trunk %
```

3. View Logs

```
trunk --zsh -- 113x50
[sheoraninfosec@Jigeshs-MacBook-Air trunk % svn update
Updating '.':
At revision 7.
[sheoraninfosec@Jigeshs-MacBook-Air trunk % svn log -v
-----
r7 | sheoraninfosec | 2026-04-04 01:13:53 +0530 (Sat, 04 Apr 2026) | 1 line
Changed paths:
   M /trunk/app.py

Version 3
-----
r6 | sheoraninfosec | 2026-04-04 01:13:35 +0530 (Sat, 04 Apr 2026) | 1 line
Changed paths:
   M /trunk/app.py

Version 2
-----
r5 | sheoraninfosec | 2026-04-04 01:13:10 +0530 (Sat, 04 Apr 2026) | 1 line
Changed paths:
   M /trunk/app.py

Version 1
-----
r4 | sheoraninfosec | 2026-04-04 01:10:27 +0530 (Sat, 04 Apr 2026) | 1 line
Changed paths:
   M /trunk/app.py

Version 3
-----
r3 | sheoraninfosec | 2026-04-04 01:09:10 +0530 (Sat, 04 Apr 2026) | 1 line
Changed paths:
   M /trunk/app.py

version 2
-----
r2 | sheoraninfosec | 2026-04-04 01:07:35 +0530 (Sat, 04 Apr 2026) | 1 line
Changed paths:
   A /trunk/app.py

Version 1
-----
r1 | sheoraninfosec | 2026-04-04 01:00:40 +0530 (Sat, 04 Apr 2026) | 1 line
Changed paths:
   A /branches
   A /tags
   A /trunk

Initial repository structure
-----
sheoraninfosec@Jigeshs-MacBook-Air trunk %
```

4. Create branch 'feature1'.

```
sheoraninfosec@Jigeshs-MacBook-Air trunk % cd ..
sheoraninfosec@Jigeshs-MacBook-Air repo_wc % svn copy file:///Users/sheoraninfosec/VCS-exp/myrepo/trunk \
file:///Users/sheoraninfosec/VCS-exp/myrepo/branches/feature1 \
-m "Created feature1 branch"
Committing transaction...
Committed revision 8.
sheoraninfosec@Jigeshs-MacBook-Air repo_wc %
```

UW PICO 5.09

File: app.py

```
print("Feature branch update")
```

5. Commit changes in branch.

```
sheoraninfosec@Jigeshs-MacBook-Air repo_wc % svn commit -m "Feature update"
sheoraninfosec@Jigeshs-MacBook-Air repo_wc %
```

6. Merge branch back to trunk.

```
sheoraninfosec@Jigeshs-MacBook-Air repo_wc % cd repo_wc/trunk
cd: no such file or directory: repo_wc/trunk
sheoraninfosec@Jigeshs-MacBook-Air repo_wc % ls
app.py      branches    feature_wc  tags        trunk
sheoraninfosec@Jigeshs-MacBook-Air repo_wc % cd trunk
sheoraninfosec@Jigeshs-MacBook-Air trunk % svn update
Updating '.':
At revision 8.
sheoraninfosec@Jigeshs-MacBook-Air trunk % svn merge file:///Users/sheoraninfosec/VCS-exp/myrepo/branches/feature
1
--- Recording mergeinfo for merge of r8 into '.':
U .
sheoraninfosec@Jigeshs-MacBook-Air trunk % svn commit -m "Merged feature1 into trunk"
Sending .
Committing transaction...
Committed revision 9.
sheoraninfosec@Jigeshs-MacBook-Air trunk %
```

Task 6 – Analysis of SVN (Centralized VCS)

1. Server Dependency

- SVN follows a **centralized client-server architecture**.
- All version history is stored on a **central repository (server)**.
- Users must connect to the server for most operations like commit, update, and log retrieval.
- If the server is unavailable:
 - Users cannot commit changes
 - Collaboration is halted
- This creates a **single point of failure**.
- During the experiment, it was observed that all commits required communication with the central repository.

2. Branching Complexity

- Branching in SVN is implemented using **directory copying (svn copy)**.
- Although efficient internally (cheap copy), it is:
 - Less intuitive compared to distributed systems
 - Requires manual handling of paths
- Branch management becomes complex as:
 - Multiple branches increase directory clutter
 - Merging requires explicit commands
- In the experiment, branch **feature1** was created and merged manually, showing that branching is not automatic or seamless.

3. History Management

- SVN maintains a **centralized and linear revision history**.
- Each commit is assigned a **global revision number** (e.g., r1, r2, r3).
- History includes:
 - Author
 - Timestamp
 - Commit message
 - Changed paths
- Easy to track project-wide changes due to single repository.
- However:
 - History is not locally available without server access
 - No full offline history like distributed systems

PART D – Distributed Version Control Using Mercurial

Task 7:

1. Initialize repository.

```
[sheoraninfosec@Jigeshs-MacBook-Air trunk % cd ..  
[sheoraninfosec@Jigeshs-MacBook-Air repo_wc % cd ..  
[sheoraninfosec@Jigeshs-MacBook-Air svn % cd ..  
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % mkdir hg_repo  
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % cd hg_repo  
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg init  
zsh: command not found: hg  
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % brew install mercurial
```

```
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg init  
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg --version  
Mercurial Distributed SCM (version 7.2.1)  
(see https://mercurial-scm.org for more information)  
  
Copyright (C) 2005-2026 Olivia Mackall and others  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo %
```

2. Add and commit Version 1.

UW PICO 5.09

File: app.py

```
print("Version 1")
```

```
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % nano app.py  
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg add app.py  
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg commit -m "Version 1"  
abort: no username supplied  
(use 'hg config --edit' to set your username)  
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg config --edit  
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg commit -m "Version 1"  
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo %
```

3. Modify and commit Version 2 and Version 3.

UW PICO 5.09

File: app.py

```
print("Version 2")
```

```
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % nano app.py  
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg commit -m "Version 2"  
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo %
```

UW PICO 5.09

File: app.py

```
print("Version 3")
```

```

[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % nano app.py
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg commit -m "Version 3"
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg log
changeset: 2:b3d67c51e787
tag: tip
user: Jigesh <jigesh@example.com>
date: Sat Apr 04 01:41:43 2026 +0530
summary: Version 3

changeset: 1:118dbcaf48f5
user: Jigesh <jigesh@example.com>
date: Sat Apr 04 01:40:55 2026 +0530
summary: Version 2

changeset: 0:5deb7f90a5a7
user: Jigesh <jigesh@example.com>
date: Sat Apr 04 01:39:08 2026 +0530
summary: Version 1

sheoraninfosec@Jigeshs-MacBook-Air hg_repo % █

```

Task 8:

1. Create branch 'feature1'.

```

[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg branch feature1
marked working directory as branch feature1
(branches are permanent and global, did you want a bookmark?)
sheoraninfosec@Jigeshs-MacBook-Air hg_repo % █

```

2. Commit feature changes.

```

UW PICO 5.09 File: app.py

print("Feature branch update")█

```

```

[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % nano app.py
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg commit -m "Feature update in feature1"
sheoraninfosec@Jigeshs-MacBook-Air hg_repo % █

```

3. View commit graph

```

[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg log --graph
@ changeset: 3:4ae660b0def
| branch: feature1
| tag: tip
| user: Jigesh <jigesh@example.com>
| date: Sat Apr 04 01:44:26 2026 +0530
| summary: Feature update in feature1
|
o changeset: 2:b3d67c51e787
| user: Jigesh <jigesh@example.com>
| date: Sat Apr 04 01:41:43 2026 +0530
| summary: Version 3
|
o changeset: 1:118dbcaf48f5
| user: Jigesh <jigesh@example.com>
| date: Sat Apr 04 01:40:55 2026 +0530
| summary: Version 2
|
o changeset: 0:5deb7f90a5a7
| user: Jigesh <jigesh@example.com>
| date: Sat Apr 04 01:39:08 2026 +0530
| summary: Version 1

sheoraninfosec@Jigeshs-MacBook-Air hg_repo % █

```

4. Clone repository

```
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % cd ..
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % hg clone hg_repo clone_repo
updating to branch default
1 files updated, 0 files merged, 0 files removed, 0 files unresolved
[sheoraninfosec@Jigeshs-MacBook-Air VCS-exp % cd clone_repo
[sheoraninfosec@Jigeshs-MacBook-Air clone_repo %
```

5. Commit in clone

```
UW PICO 5.09 File: app.py

print("Change from cloned repo")
```

```
[sheoraninfosec@Jigeshs-MacBook-Air clone_repo % nano app.py
[sheoraninfosec@Jigeshs-MacBook-Air clone_repo % hg commit -m "Update from clone"
[sheoraninfosec@Jigeshs-MacBook-Air clone_repo %
```

6. Push to original repo

```
[sheoraninfosec@Jigeshs-MacBook-Air clone_repo % hg push ../hg_repo
pushing to ../hg_repo
searching for changes
adding changesets
adding manifests
adding file changes
added 1 changesets with 1 changes to 1 files (+1 heads)
[sheoraninfosec@Jigeshs-MacBook-Air clone_repo % cd ../hg_repo

[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg pull ../clone_repo
pulling from ../clone_repo
searching for changes
no changes found
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg update
0 files updated, 0 files merged, 0 files removed, 0 files unresolved
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo % hg log --graph
o changeset: 4:75fcd81b6447
| tag: tip
| parent: 2:b3d67c51e787
| user: Jigesh <jigesh@example.com>
| date: Sat Apr 04 01:47:19 2026 +0530
| summary: Update from clone
|
| @ changeset: 3:4aeee60b0def
| / branch: feature1
| user: Jigesh <jigesh@example.com>
| date: Sat Apr 04 01:44:26 2026 +0530
| summary: Feature update in feature1
|
o changeset: 2:b3d67c51e787
| user: Jigesh <jigesh@example.com>
| date: Sat Apr 04 01:41:43 2026 +0530
| summary: Version 3
|
o changeset: 1:118dbcaf48f5
| user: Jigesh <jigesh@example.com>
| date: Sat Apr 04 01:40:55 2026 +0530
| summary: Version 2
|
o changeset: 0:5deb7f90a5a7
| user: Jigesh <jigesh@example.com>
| date: Sat Apr 04 01:39:08 2026 +0530
| summary: Version 1
|
[sheoraninfosec@Jigeshs-MacBook-Air hg_repo %
```

Task 9 – Analysis of Mercurial (Distributed VCS)

1. Distributed Architecture

- Mercurial follows a **distributed architecture**, where every user has a **complete copy of the repository**.
- Each clone contains:
 - Full commit history
 - All branches
 - File versions
- There is **no central dependency** — repositories can sync using push/pull.

2. Offline Capability

- All operations such as:
 - Commit
 - Branch
 - Log viewingcan be performed **without internet or server access**.
- Changes are stored locally and can be pushed later.
- This shows **true offline workflow support**, unlike SVN.

3. Merge Handling

- Mercurial supports **automatic and efficient merging**.
- Uses a **Directed Acyclic Graph (DAG)** to manage history.
- Multiple heads can exist (parallel development lines).
- Conflicts (if any) are handled with built-in merge tools.
- Merge operations are faster and less complex than SVN.

4. Advantages

- Full repository available locally
- Supports offline work
- Fast commits and operations
- Efficient branching and merging
- No single point of failure
- Ideal for distributed teams and open-source projects

Task 10 – Comparative Study of VCS

Comparison Table

Feature	RCS (Local VCS)	SVN (Centralized VCS)	Mercurial (Distributed VCS)
Architecture Model	Local (single-user)	Centralized client-server	Distributed (peer-to-peer)
Repository Structure	File-based (RCS/.v files)	Central repository with trunk, branches, tags	Full repository copy in each system
Collaboration Support	No collaboration	Multi-user (via server)	Strong distributed collaboration
Offline Capability	(local only)	Requires server	Full offline support
Branching Efficiency	Not supported	Complex and manual	Fast and efficient
Merge Handling	Not supported	Manual and error-prone	Automatic and efficient
Scalability	Very low	Moderate	High
Security Considerations	Low (no access control)	Medium (centralized control)	High (distributed + integrity checks)
Suitability – Academic Research	Basic individual work	Moderate team projects	Best for collaborative research
Suitability – Enterprise Systems	Not suitable	Controlled environments	Large-scale systems
Suitability – Open Source Projects	Not suitable	Limited	Ideal

PART F – Critical Thinking Answers

1. Why did distributed VCS become dominant in DevOps environments?

- Supports **parallel development** with multiple contributors working independently
- Enables **fast local commits** without server dependency
- Works seamlessly with **CI/CD pipelines** due to easy branching and merging
- Provides **high availability** (no single point of failure)
- Allows **offline development**, increasing productivity
- Better suited for **microservices and rapid deployment cycles**
- In the experiment, cloning and pushing changes demonstrated real distributed workflow

2. Compare conflict resolution in SVN and Mercurial

Aspect	SVN	Mercurial
Conflict Handling	Manual and rigid	More flexible and automated
Workflow	Centralized → conflicts during commit/update	Distributed → conflicts during merge
Merge Complexity	High	Lower
Tools	Limited	Better built-in support

3. Which model is most suitable for CI/CD pipelines and why?

Distributed VCS (Mercurial / Git)

Reasons:

- Fast branching and merging
- Supports automated workflows
- Works well with build systems and pipelines
- No dependency on central server for commits
- Enables continuous integration and rapid deployment

4. How does repository architecture affect distributed team performance?

- **Centralized (SVN):**
 - Slower due to server dependency
 - Bottlenecks when multiple users access server
 - Limited offline work
- **Distributed (Mercurial):**
 - Faster development cycles
 - Parallel work without conflicts
 - Better collaboration across locations
 - Reduced downtime impact

Result: Distributed architecture significantly improves team productivity and scalability.

5. If designing a new VCS today, what architectural improvements would you introduce?

- Hybrid model (centralized + distributed benefits)
- Built-in **AI-based conflict resolution**
- Enhanced **security with tamper-proof logs (blockchain-like integrity)**
- Real-time collaboration (like Google Docs for code)
- Smarter branching visualization tools
- Automated dependency tracking
- Integrated CI/CD pipeline support

-----**END OF DOCUMENT**-----